



金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析

1. 往期部分专题

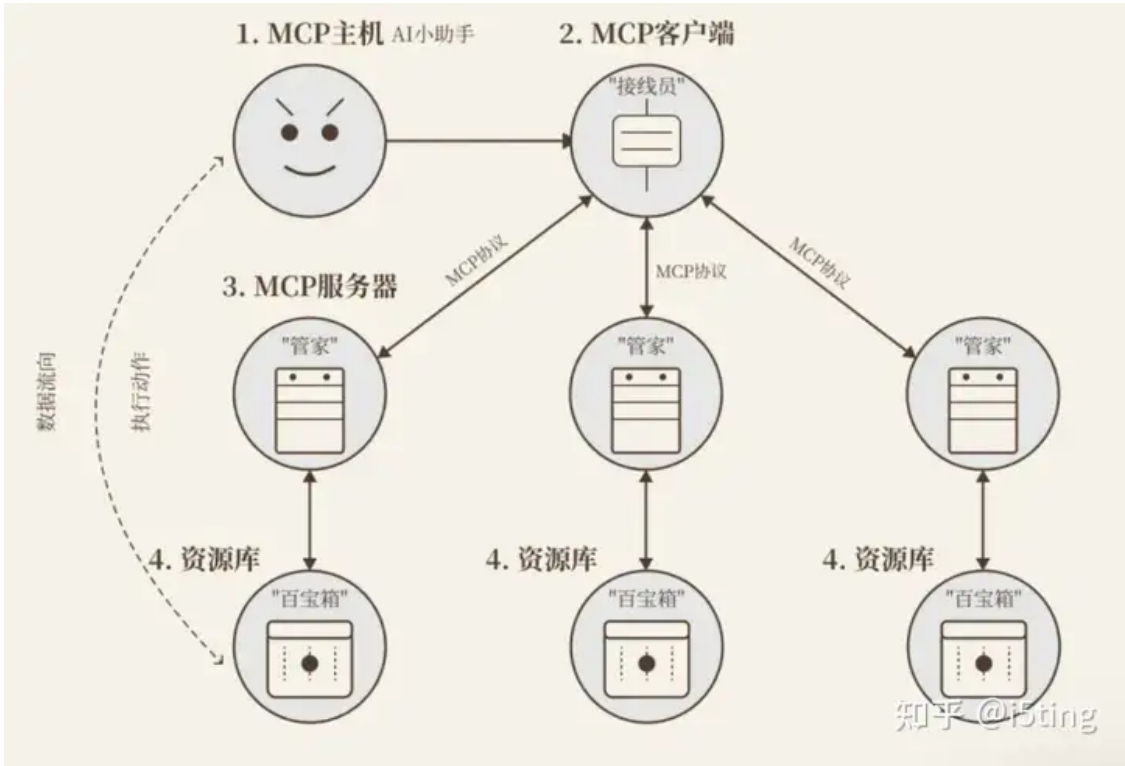
- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能 Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)

2. Ai浪潮下前端后续的发展是怎样的？

AI Agent 发展互联中，前端是第一批受益者

2.1 前端转型做agent应用开发是主流

前端擅长做应用和交互，最合适的工作的就是做ai agent应用，这个事儿一定会火起来。目前ai基建能卷的，都不是小公司了。但做agent应用，才开始。比如[smithery](#)这个mcp源上才1000多个mcp服务。[cline](#)的mcp市场也刚刚开始。mcp解决了api式服务的规范问题。（mcp现在特别粗糙，都是计划）



lightpanda在ai无头浏览器性能也是chrome的11倍，兼容playwrite，Puppeteer（cdp模式）等。通过无头浏览器做自动已经具备可行性。

所以开发agent应用是主流。

- 基建可用。会出新入口，比如smithery，会改变交互，甚至是颠覆。技术难度会比较高
- 业务可用。技术难度低。只要有想法，就可以像发npm包一样提供agent服务，按需收费。

2.2 技术部分探索

对于技术栈，摘自我之前的回答，还比较浅，入门够了。

任何时代 和用户交互 都是避免不了的，交互变革也会随着agent变多和管理而改变，又是一波新机会。这里通过5步开源项目帮大家把技术串联起来。

1. ai入门：ai-chatbot

为了学习，和探索，可以研究了一下<https://github.com/vercel/ai-chatbot>，这是一个非常好的易于学习的Node.js AI Agent开发项目模版，各种最新的技术栈基本都覆盖了。

- a. 基于next，ai-sdk，实现基于openai模型的chatbot功能，技术栈和功能很实用。
- b. 足够小，100个ts文件，代码行数10000行左右，结构清晰，简单易学。
- c. 前后端，典型的全栈应用，包含postgre db，鉴权，react，ai，非常全面了。很多最佳实践，比如toast库，密码加盐处理等。
- d. 可以使用aihubmix等openai代理服务，支持支付宝。
- e. 扩展性强，使用其他模型，以及langchain、llamaindex等。

部署还是有点麻烦，涉及的内容还是比较多的。

学完这个，你发现很多网站，网页，都是这个上面缝缝补补的。

2. ai搜索：scira

这个开源项目也很简单，非常适合入门。如果已经熟悉了ai-sdk，这里再学一些搜索服务集成（比如Tavily AI），也是极好的。它在应用层面上，是非常简单且实用的示例。

3. 深挖基础lib：langchain 和 llamaindex

深挖langchain和llamaindex，没啥好说的，这是ai世界里，构建rag必备的包，都是现有py版本，然后才有js/ts版本，可见在应用开发领域，js/ts依然是最受欢迎的方式。

4. 探索客户端，基于vite + electron-forge，另外还有很多rust实现

<https://github.com/block/goose>

其中agent、扩展，以及类似于Eventloop的机制，有很多很有意思的实现。

5. 探索应用开发。上面的技术基本上都包含了，比如llamaindex

<https://github.com/mastra-ai/mastra>

基本上把各种ai sdk都集成了，agent、tool、workflow、rag等全部支持，开箱即用，写起来更简单。

3. 前端大厂面试场景题原题解析

3.1 字节

3.1.1 两数之和

给定一个整数数组 `nums` 和一个目标值 `target`，请你在该数组中找出和为目标值的那 **两个** 整数，并返回他们的数组下标。

你可以假设每种输入只会对应一个答案。但是，你不能重复利用这个数组中同样的元素。

示例:

Code block

```
1  给定 nums = [2, 7, 11, 15], target = 9
2  因为 nums[0] + nums[1] = 2 + 7 = 9所以返回 [0, 1]
```

解题思路:

- 初始化一个 `map = new Map()`
- 从第一个元素开始遍历 `nums`
- 获取目标值与 `nums[i]` 的差值，即 `k = target - nums[i]`，判断差值在 `map` 中是否存在
 - 不存在（`map.has(k)` 为 `false`），则将 `nums[i]` 加入到 `map` 中（key为 `nums[i]`，value为 `i`，方便查找 `map`中是否存在某值，并可以通过 `get` 方法直接拿到下标）
 - 存在（`map.has(k)`），返回 `[map.get(k), i]`，求解结束
- 遍历结束，则 `nums` 中没有符合条件的两个数，返回 `[]`

时间复杂度: $O(n)$

代码实现:

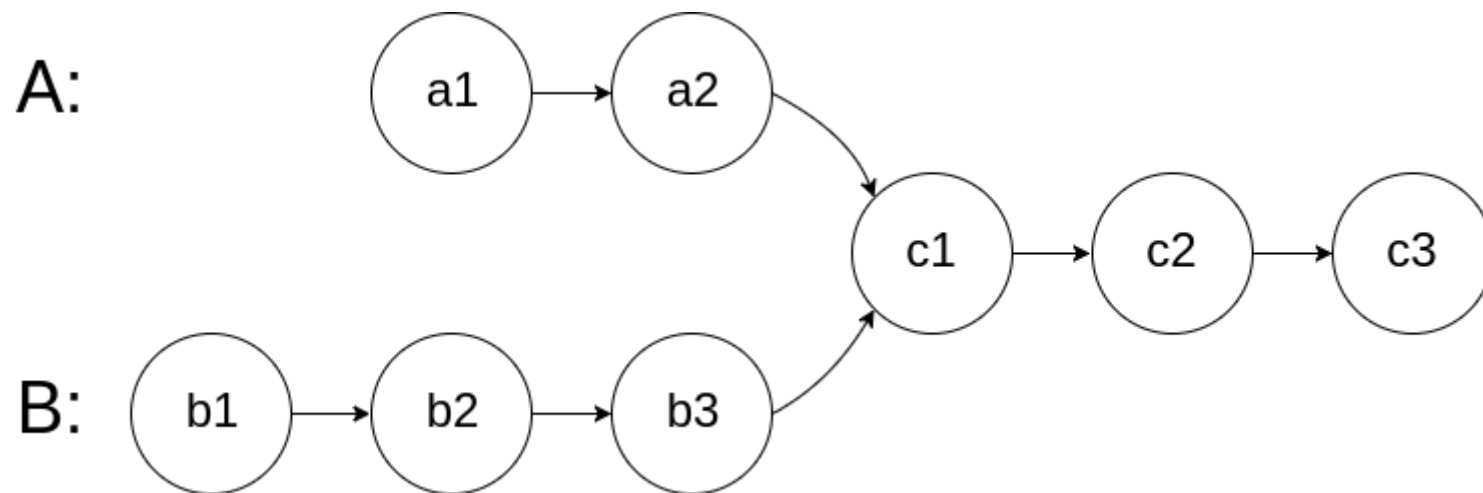
Code block

```
1  const twoSum = function(nums, target) {
2    let map = new Map()
3    for(let i = 0; i < nums.length; i++) {
4      let k = target-nums[i]
5      if(map.has(k)) {
6        return [map.get(k), i]
7      }
8      map.set(nums[i], i)
9    }
10   return [];
11  };
```

3.1.2 编写一个程序，找到两个单链表相交的起始节点

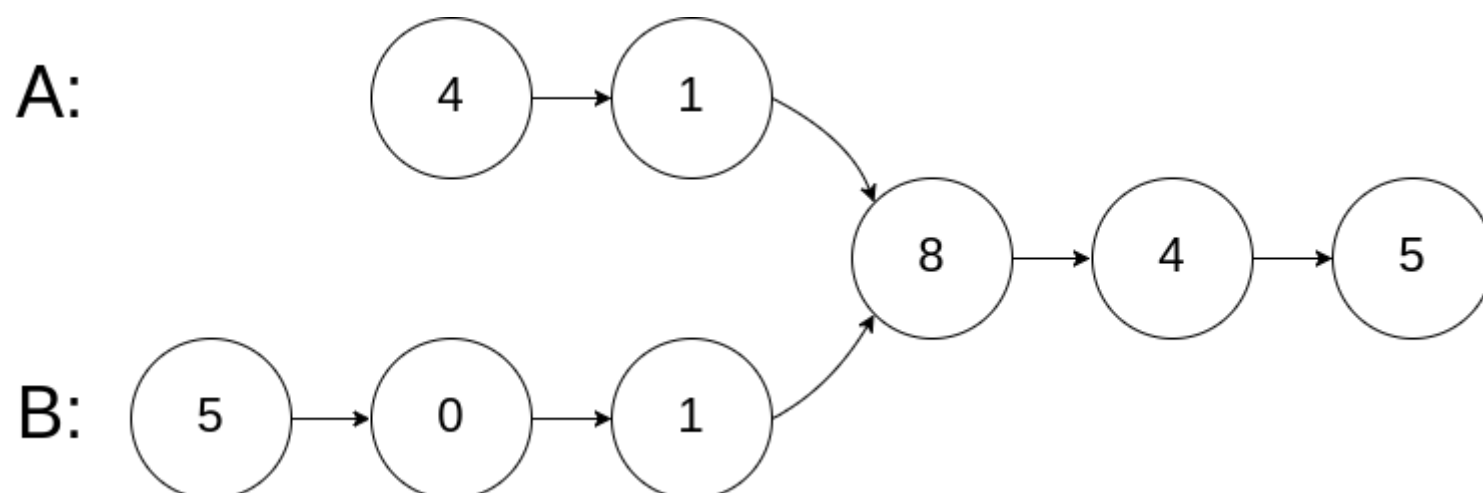
编写一个程序，找到两个单链表相交的起始节点。

如下面的两个链表：



在节点 c1 开始相交。

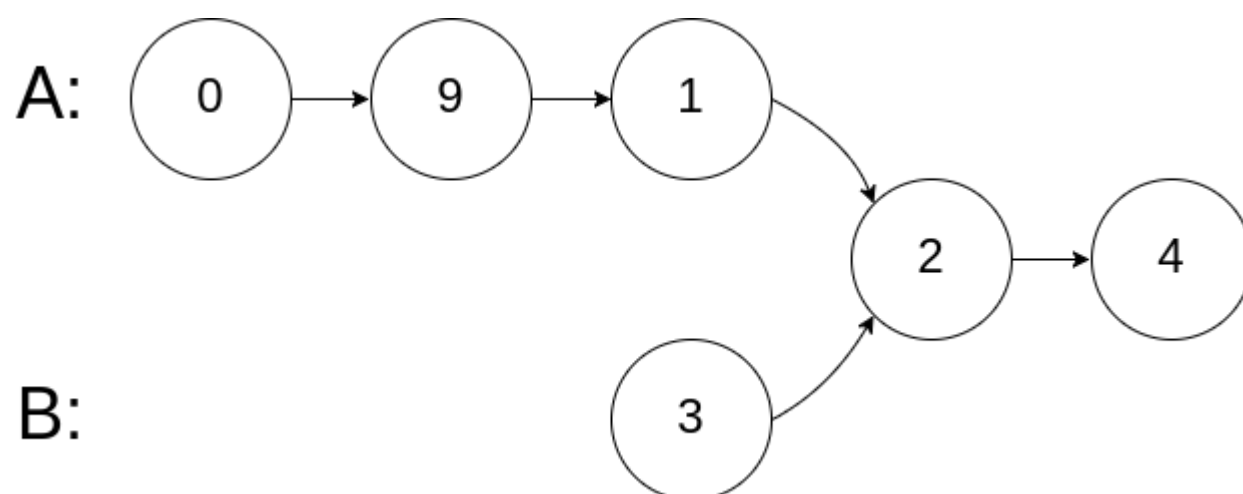
示例 1:



Code block

- 1 输入: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3
- 2 输出: Reference of the node with value = 8
- 3 输入解释: 相交节点的值为 8 (注意, 如果两个列表相交则不能为 0)。从各自的表头开始算起, 链表 A 为 [4,1,8,4,5], 链表 B 为 [5,0,1,8,4,5]。在 A 中, 相交节点前有 2 个节点; 在 B 中, 相交节点前有 3 个节点。

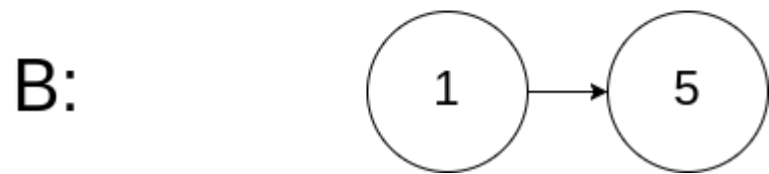
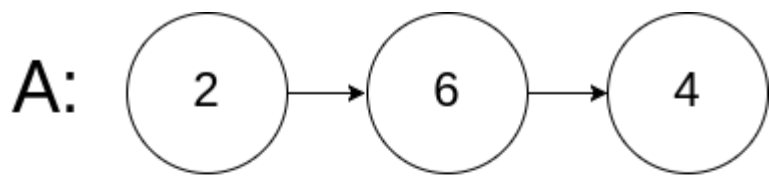
示例 2:



Code block

- 1 输入: intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
- 2 输出: Reference of the node with value = 2
- 3 输入解释: 相交节点的值为 2 (注意, 如果两个列表相交则不能为 0)。从各自的表头开始算起, 链表 A 为 [0,9,1,2,4], 链表 B 为 [3,2,4]。在 A 中, 相交节点前有 3 个节点; 在 B 中, 相交节点前有 1 个节点。

示例 3:



Code block

```
1  输入: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
2  输出: null
3  输入解释: 从各自的表头开始算起, 链表 A 为 [2,6,4], 链表 B 为 [1,5]。由于这两个链表不相交, 所以 intersectVal 必须为 0, 而 skipA 和 skipB 可以是任意值。
4  解释: 这两个链表不相交, 因此返回 null。
```

注意:

- 如果两个链表没有交点, 返回 null.
- 在返回结果后, 两个链表仍须保持原有的结构。
- 可假定整个链表结构中没有循环。
- 程序尽量满足 $O(n)$ 时间复杂度, 且仅用 $O(1)$ 内存。

方法: 遍历

- 先将两个链表转换成数组结构
- 对其中一个数组进行遍历, 每次遍历判断另一个数组是否含有该值
- 遍历结束后如果没有找到就返回 null

Code block

```
1  function intersectNode( head1, head2 ) {
2      var arr1 = [],
3          arr2 = [],
4          theValue = null;
5      transformToArray( head1, arr1 );
6      transformToArray( head2, arr2 );
7
8      for (var i = 0; i < arr1.length; i++) {
9          if( arr2.indexOf(arr1[i]) !== -1 ){
10             theValue = arr1[i];
11             return 'Reference of the node with value = ' + theValue;
12         }
13     }
14     return theValue;
15
16     /*
17         arr1.some( value => {
18             if ( arr2.includes(value) ){
19                 theValue = value;
20                 return true;
21             }
22         })
23     if( theValue ){
```

```

24         return 'Reference of the node with value = ' + theValue;
25     }
26     return null;
27     */
28 }
29
30 function transformToArray( head, arr ) {
31     while( head ){
32         arr.push(head.val);
33         head = head.next;
34     }
35 }

```

测试代码：

Code block

```

1  function CreateNode(val){
2      this.val = val;
3      this.next = null;
4  }
5  function CreateList(...nodes){
6      this.head = nodes[0];
7      this.length = nodes.length;
8      for( var i = 0; i < nodes.length - 1; i++ ){
9          if( nodes[i+1] ){
10             nodes[i].next = nodes[i+1];
11         }
12     }
13 }
14
15 function intersectNode( head1, head2 ) {
16     var arr1 = [],
17         arr2 = [],
18         theValue = null;
19     transformToArray( head1, arr1 );
20     transformToArray( head2, arr2 );
21
22     for (var i = 0; i < arr1.length; i++) {
23         if( arr2.indexOf(arr1[i]) !== -1 ){
24             theValue = arr1[i];
25             return 'Reference of the node with value = ' + theValue;
26         }
27     }
28     return theValue;
29 }
30 function transformToArray( head, arr ) {
31     while( head ){
32         arr.push(head.val);
33         head = head.next;
34     }
35 }
36
37 const node1 = new CreateNode(1);
38 const node2 = new CreateNode(2);
39 const node3 = new CreateNode(3);
40 const node4 = new CreateNode(4);
41 const node5 = new CreateNode(5);
42 const nodeA = new CreateNode('A');
43 const nodeB = new CreateNode('B');
44 const nodeC = new CreateNode('C');

```

```
45  const nodeD = new CreateNode('D');
46  const nodeE = new CreateNode('E');
47  const list1 = new CreateList(node1, node2, node3, node4, node5);
48  const list2 = new CreateList(nodeA, nodeB, node3, nodeC, nodeD);
49
50  intersectNode(list1.head, list2.head);
```

3.1.3 翻转字符串里的单词

给定一个字符串，逐个翻转字符串中的每个单词。

示例 1：

Code block

```
1  输入: "the sky is blue"
2  输出: "blue is sky the"
```

示例 2：

Code block

```
1  输入: "  hello world!  "
2  输出: "world! hello"
3  解释: 输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。
```

示例 3：

Code block

```
1  输入: "a good   example"
2  输出: "example good a"
3  解释: 如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。
```

说明：

- 无空格字符构成一个单词。
- 输入字符串可以在前面或者后面包含多余的空格，但是反转后的字符不能包括。
- 如果两个单词间有多余的空格，将反转后单词间的空格减少到只含一个。

解法一：正则 + JS API

Code block

```
1  var reverseWords = function(s) {
2      return s.trim().replace(/\s+/g, ' ').split(' ').reverse().join(' ');
3  };
```

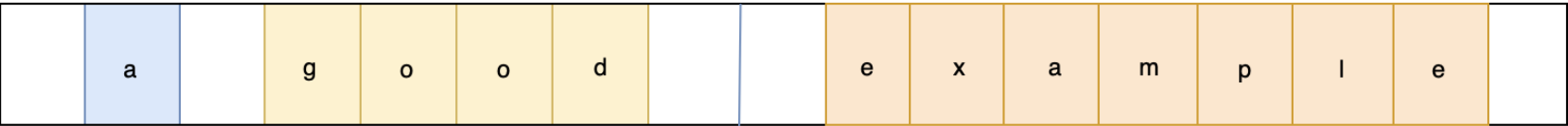
解法二：双端队列（不使用 API）

双端队列，故名思义就是两端都可以进队的队列

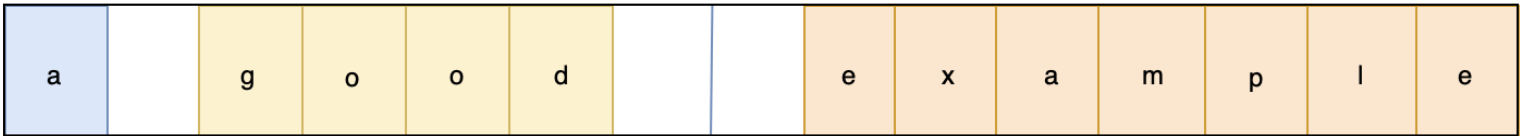
解题思路：

- 首先去除字符串左右空格
- 逐个读取字符串中的每个单词，依次放入双端队列的对头
- 再将队列转换成字符串输出（以空格为分隔符）

画图理解：



删除字符串两端的空格



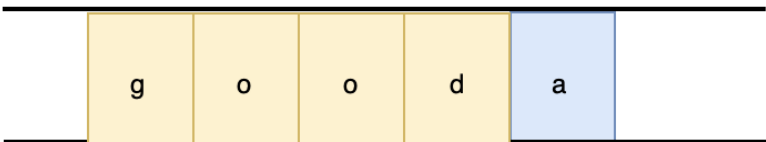
从左到右，依次读取单词



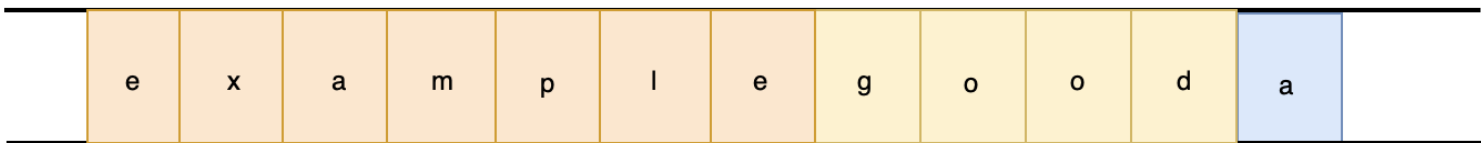
将第一个单词放入双端队列队头中



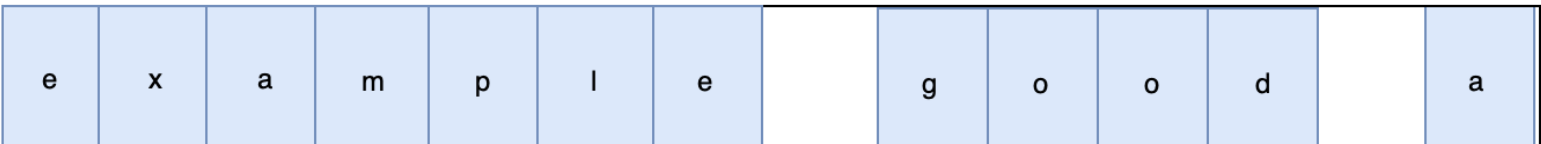
将第二个单词也双端队列队头中



依次，将所有的单词都放入双端队列队头中
此时，队列中的单词顺序被翻转



将队列中的单词以字符串的形式输出
并以空格作为分隔符



代码实现：

Code block

```
1  var reverseWords = function(s) {
2      let left = 0
3      let right = s.length - 1
4      let queue = []
5      let word = ''
6      while (s.charAt(left) === ' ') left ++
7      while (s.charAt(right) === ' ') right --
8      while (left <= right) {
9          let char = s.charAt(left)
10         if (char === ' ' && word) {
11             queue.unshift(word)
12             word = ''
13         } else if (char !== ' '){
14             word += char
15         }
16         left++
17     }
18     queue.unshift(word)
19     return queue.join(' ')
20 };
```

3.1.4 无重复字符的最长子串

示例 1:

Code block

```
1  输入: "abcabcbb"
2  输出: 3
3  解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。
```

示例 2:

Code block

```
1  输入: "bbbbbb"
2  输出: 1
3  解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。
```

示例 3:

Code block

```
1  输入: "pwwkew"
2  输出: 3
3  解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。请注意, 你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。
```

快慢指针维护一个滑动窗口，如果滑动窗口里面有快指针fastp所指向的元素（记录这重复元素在滑动窗口的索引tmp），那么滑动窗口要缩小，即慢指针slowp要移动到tmp的后一个元素。

Code block

```
1  /**
2   * @param {string} s
```

```

3    * @return {number}
4    */
5    var lengthOfLongestSubstring = function(s) {
6        let res = 0, slowp = 0, tmp = 0
7        for(let fastp = 0; fastp < s.length; fastp++) {
8            tmp = s.substring(slowp, fastp).indexOf(s.charAt(fastp)) // 滑动窗口有没有重复元素
9            if(tmp === -1) { // 没有
10                res = Math.max(res, fastp-slowp+1) // 上一次值和滑动窗口值大小比较
11            }else{ // 有, 移动慢指针, 是slowp+tmp+1不是tmp+1, 因为tmp是根据滑动窗口算的
12                slowp = slowp + tmp + 1
13            }
14        }
15        return res
16    };

```

3.1.5 无重复字符的最长子串

给定一个字符串，请你找出其中不含有重复字符的 **最长子串** 的长度。

示例 1:

Code block

```

1    输入: "abcabcbb"
2    输出: 3
3    解释: 因为无重复字符的最长子串是 "abc", 所以其长度为 3。

```

示例 2:

Code block

```

1    输入: "bbbbbb"
2    输出: 1
3    解释: 因为无重复字符的最长子串是 "b", 所以其长度为 1。

```

示例 3:

Code block

```

1    输入: "pwwkew"
2    输出: 3
3    解释: 因为无重复字符的最长子串是 "wke", 所以其长度为 3。请注意，你的答案必须是 子串 的长度, "pwke" 是一个子序列, 不是子串。

```

快慢指针维护一个滑动窗口，如果滑动窗口里面有快指针fastp所指向的元素（记录这重复元素在滑动窗口的索引tmp），那么滑动窗口要缩小，即慢指针slowp要移动到tmp的后一个元素。

Code block

```

1    /**
2     * @param {string} s
3     * @return {number}
4     */
5    var lengthOfLongestSubstring = function(s) {
6        let res = 0, slowp = 0, tmp = 0
7        for(let fastp = 0; fastp < s.length; fastp++) {
8            tmp = s.substring(slowp, fastp).indexOf(s.charAt(fastp)) // 滑动窗口有没有重复元素
9            if(tmp === -1) { // 没有
10                res = Math.max(res, fastp-slowp+1) // 上一次值和滑动窗口值大小比较

```

```

11     }else{ // 有，移动慢指针， 是slowp+tmp+1不是tmp+1，因为tmp是根据滑动窗口算的
12         slowp = slowp + tmp + 1
13     }
14 }
15 return res
16 };

```

3.2 阿里

3.2.1 链表求和

给定两个用链表表示的整数，每个节点包含一个数位。

这些数位是反向存放的，也就是个位排在链表首部。

编写函数对这两个整数求和，并用链表形式返回结果。

示例：

Code block

```

1  输入：(7 -> 1 -> 6) + (5 -> 9 -> 2)，即617 + 295
2  输出：2 -> 1 -> 9，即912
3  进阶：思考一下，假设这些数位是正向存放的，又该如何解决呢？

```

示例：

Code block

```

1  输入：(6 -> 1 -> 7) + (2 -> 9 -> 5)，即617 + 295
2  输出：9 -> 1 -> 2，即912

```

用carry存储每次的进位

Code block

```

1  var addTwoNumbers = function(l1, l2) {
2      let carry = 0;
3      let root = new ListNode(0)
4      let p = root;
5      while (l1 || l2) {
6          let sum = 0
7          if (l1) {
8              sum += l1.val
9              l1 = l1.next
10         }
11         if (l2) {
12             sum += l2.val
13             l2 = l2.next
14         }
15         sum += carry
16         carry = Math.floor(sum / 10)
17         p.next = new ListNode(sum % 10)
18         p = p.next
19     }
20     if (carry === 1) {
21         p.next = new ListNode(carry)
22         p = p.next
23     }
24     return root.next
25 };

```

3.2.2 二叉树的层次遍历

给定一个二叉树，返回其节点值自底向上的层次遍历。（即按从叶子节点所在层到根节点所在的层，逐层从左向右遍历）

例如：

给定二叉树 `[3,9,20,null,null,15,7]`，

Code block

```
1      3
2     / \
3    9  20
4   /  \
5  15   7
```

返回其自底向上的层次遍历为：

Code block

```
1  [
2   [15,7],
3   [9,20],
4   [3]
5  ]
```

使用一个嵌套队列，外层队列保存每层的数据，内层的队列保存当前层所有的节点

Code block

```
1  var levelOrderBottom = function (root) {
2    if (root === null) return [];
3    let queue = [[root]]; // 外层队列
4    let print = [];
5    while (queue.length) {
6      let currentLevel = queue.shift(); // 当前遍历的层，也是一个队列保存的是当前层所有的节点
7      let currentLevelVal = [];
8      let nextLevel = []; // 准备保存下一层所有的节点
9      while (currentLevel.length) {
10       let currentNode = currentLevel.shift(); // 遍历当前
11       currentLevelVal.push(currentNode.val);
12
13       // 根据当前层节点是否有子节点构造下一层的队列
14       if (currentNode.left !== null) {
15         nextLevel.push(currentNode.left);
16       }
17       if (currentNode.right !== null) {
18         nextLevel.push(currentNode.right);
19       }
20     }
21
22     // 如果下一层队列还有节点，加到外层队列
23     if (nextLevel.length) {
24       queue.push(nextLevel);
25     }
26     if (currentLevel.length === 0) {
27       print.unshift(currentLevelVal);
28     }
29   }
30 }
```

```
31     return print;
32 };
```

- 复杂度分析

时间复杂度：每个节点都会被遍历一次，因此时间复杂度是 $O(n)$

空间复杂度：需要存储整棵树的值，因此空间复杂度是 $O(n)$

3.3 腾讯

3.3.1 字符串相乘

给定两个以字符串形式表示的非负整数 num1 和 num2，返回 num1 和 num2 的乘积，它们的乘积也表示为字符串形式。

示例 1:

Code block

```
1  输入：num1 = "2", num2 = "3"
2  输出："6"
```

示例 2:

Code block

```
1  输入：num1 = "123", num2 = "456"
2  输出："56088"
```

说明:

- num1 和 num2 的长度小于110。
- num1 和 num2 只包含数字 0-9。
- num1 和 num2 均不以零开头，除非是数字 0 本身。
- 不能使用任何标准库的大数类型（比如 BigInteger）或直接将输入转换为整数来处理。

解法一：常规解法

从右往左遍历乘数，将乘数的每一位与被乘数相乘得到对应的结果，再将每次得到的结果累加

另外，当乘数的每一位与被乘数高位（非最低位）相乘的时候，注意低位补 '0'

Code block

```
1  let multiply = function(num1, num2) {
2      if (num1 === "0" || num2 === "0") return "0"
3
4      // 用于保存计算结果
5      let res = "0"
6
7      // num2 逐位与 num1 相乘
8      for (let i = num2.length - 1; i >= 0; i--) {
9          let carry = 0
10         // 保存 num2 第i位数字与 num1 相乘的结果
11         let temp = ''
12         // 补 0
13         for (let j = 0; j < num1.length - 1 - i; j++) {
14             temp+='0'
15         }
16         let n2 = num2.charAt(i) - '0'
```

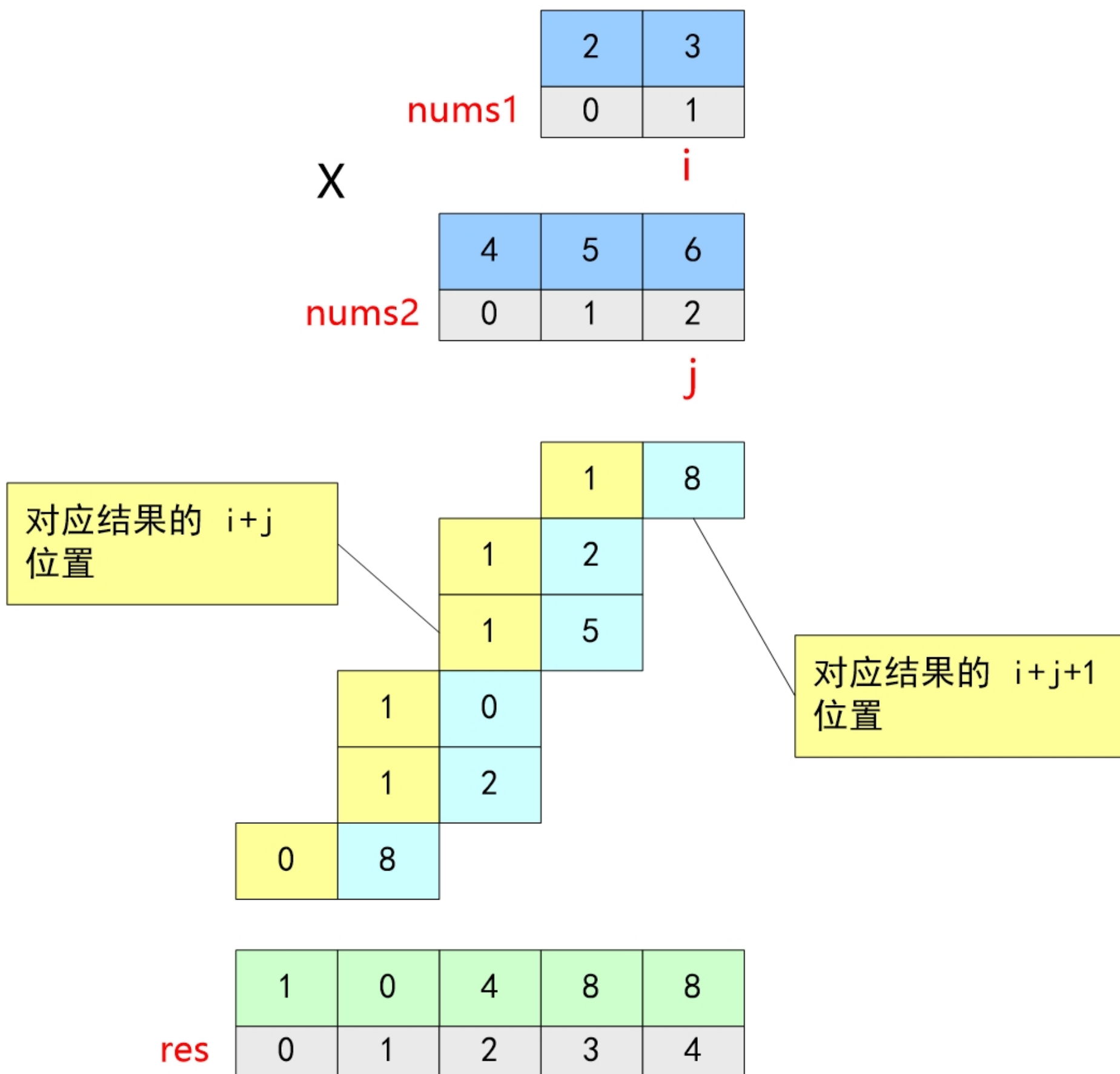
```
17
18      // num2 的第 i 位数字 n2 与 num1 相乘
19      for (let j = num1.length - 1; j >= 0 || carry != 0; j--) {
20          let n1 = j < 0 ? 0 : num1.charAt(j) - '0'
21          let product = (n1 * n2 + carry) % 10
22          temp += product
23          carry = Math.floor((n1 * n2 + carry) / 10)
24      }
25      // 将当前结果与新计算的结果求和作为新的结果
26      res = addStrings(res, Array.prototype.slice.call(temp).reverse().join(""))
27  }
28  return res
29 }
30
31 let addStrings = function(num1, num2) {
32     let a = num1.length, b = num2.length, result = '', tmp = 0
33     while(a || b) {
34         a ? tmp += +num1[--a] : ''
35         b ? tmp += +num2[--b] : ''
36
37         result = tmp % 10 + result
38         if(tmp > 9) tmp = 1
39         else tmp = 0
40     }
41     if (tmp) result = 1 + result
42     return result
43 }
```

复杂度分析：

- 时间复杂度： $O(\max(m \cdot n, n \cdot n))$
- 空间复杂度： $O(m+n)$

解法二：竖式相乘（优化）

两个数M和N相乘的结果可以由 **M 乘上 N 的每一位数的和得到**，如下图所示：



- 计算 `num1` 依次乘上 `num2` 的每一位的和
- 把得到的所有和按对应的位置累加在一起, 就可以得到 `num1 * num2` 的结果

Code block

```
1  let multiply = function(num1, num2) {
2    if(num1 === '0' || num2 === '0') return "0"
3
4    // 用于保存计算结果
5    let res = []
6
7    // 从个位数开始逐位相乘
8    for(let i = 0; i < num1.length; i++){
9      // num1 尾元素
10     let tmp1 = +num1[num1.length-1-i]
11
12     for(let j = 0; j < num2.length; j++){
13       // num2尾元素
14       let tmp2 = +num2[num2.length-1-j]
15
```

```
16          // 判断结果集索引位置是否有值
17          let pos = res[i+j] ? res[i+j]+tmp1*tmp2 : tmp1*tmp2
18          // 赋值给当前索引位置
19          res[i+j] = pos%10
20          // 是否进位 这样简化res去除不必要的"0"
21          pos >=10 && (res[i+j+1]=res[i+j+1] ? res[i+j+1]+Math.floor(pos/10) : Math.floor(pos/10));
22      }
23  }
24  return res.reverse().join("");
25 }
```

复杂度分析：

- 时间复杂度：O(m * n)
- 空间复杂度：O(m + n)

3.3.2 有效的括号

给定一个只包括 '(' ， ')' ， '{' ， '}' ， '[' ， ']' 的字符串，判断字符串是否有效。

有效字符串需满足：

- 左括号必须用相同类型的右括号闭合。
- 左括号必须以正确的顺序闭合。

注意空字符串可被认为是有效字符串。

示例 1:

Code block

```
1  输入: "()"
2  输出: true
```

示例 2:

Code block

```
1  输入: "()[]{}"
2  输出: true
```

示例 3:

Code block

```
1  输入: "]"
2  输出: false
```

示例 4:

Code block

```
1  输入: "([)]"
2  输出: false
```

示例 5:

Code block

```
1  输入: "{[]}"
```

2 输出: true

Code block

```
1  /**
2   * @param {string} s
3   * @return {boolean}
4   */
5  var isValid = function(s) {
6
7      const brackets = []
8
9      if (s % 2) return false
10
11     for (const i of s) {
12         switch(i) {
13             case '{':
14                 brackets.push(i)
15                 break
16             case '[':
17                 brackets.push(i)
18                 break
19             case '(':
20                 brackets.push(i)
21                 break
22             case '}':
23                 if (brackets.pop() !== '{') return false
24                 break
25             case ']':
26                 if (brackets.pop() !== '[') return false
27                 break
28             case ')':
29                 if (brackets.pop() !== '(') return false
30                 break
31         }
32     }
33     return brackets.length === 0
34 };
```

3.4 华为

3.4.1 LRU

运用你所掌握的数据结构，设计和实现一个 LRU (最近最少使用) 缓存机制。它应该支持以下操作： 获取数据 `get` 和写入数据 `put` 。

获取数据 `get(key)` - 如果密钥 (`key`) 存在于缓存中，则获取密钥的值（总是正数），否则返回 `-1` 。

写入数据 `put(key, value)` - 如果密钥不存在，则写入数据。当缓存容量达到上限时，它应该在写入新数据之前删除最久未使用的数据，从而为新数据留出空间。

进阶:

你是否可以在 **O(1)** 时间复杂度内完成这两种操作？

示例:

Code block

```
1  LRUCache cache = new LRUCache( 2 /* 缓存容量 */ );
2
3  cache.put(1, 1);
```

```
4  cache.put(2, 2);
5  cache.get(1);      // 返回 1
6  cache.put(3, 3);    // 该操作会使得密钥 2 作废
7  cache.get(2);      // 返回 -1 (未找到)
8  cache.put(4, 4);    // 该操作会使得密钥 1 作废
9  cache.get(1);      // 返回 -1 (未找到)
10 cache.get(3);      // 返回 3
11 cache.get(4);      // 返回 4
```

基础解法：数组+对象实现

类 vue keep-alive 实现

Code block

```
1  var LRUCache = function(capacity) {
2    this.keys = []
3    this.cache = Object.create(null)
4    this.capacity = capacity
5  };
6
7  LRUCache.prototype.get = function(key) {
8    if(this.cache[key]) {
9      // 调整位置
10     remove(this.keys, key)
11     this.keys.push(key)
12     return this.cache[key]
13   }
14   return -1
15 };
16
17 LRUCache.prototype.put = function(key, value) {
18   if(this.cache[key]) {
19     // 存在即更新
20     this.cache[key] = value
21     remove(this.keys, key)
22     this.keys.push(key)
23   } else {
24     // 不存在即加入
25     this.keys.push(key)
26     this.cache[key] = value
27     // 判断缓存是否已超过最大值
28     if(this.keys.length > this.capacity) {
29       removeCache(this.cache, this.keys, this.keys[0])
30     }
31   }
32 };
33
34 // 移除 key
35 function remove(arr, key) {
36   if (arr.length) {
37     const index = arr.indexOf(key)
38     if (index > -1) {
39       return arr.splice(index, 1)
40     }
41   }
42 }
43
44 // 移除缓存中 key
45 function removeCache(cache, keys, key) {
46   cache[key] = null
```

```
47     remove(keys, key)
48 }
```

进阶：Map

利用 Map 既能保存键值对，并且能够记住键的原始插入顺序

Code block

```
1  var LRUCache = function(capacity) {
2    this.cache = new Map()
3    this.capacity = capacity
4  }
5
6  LRUCache.prototype.get = function(key) {
7    if (this.cache.has(key)) {
8      // 存在即更新
9      let temp = this.cache.get(key)
10     this.cache.delete(key)
11     this.cache.set(key, temp)
12     return temp
13   }
14   return -1
15 }
16
17 LRUCache.prototype.put = function(key, value) {
18   if (this.cache.has(key)) {
19     // 存在即更新（删除后加入）
20     this.cache.delete(key)
21   } else if (this.cache.size >= this.capacity) {
22     // 不存在即加入
23     // 缓存超过最大值，则移除最近没有使用的
24     this.cache.delete(this.cache.keys().next().value)
25   }
26   this.cache.set(key, value)
27 }
```

3.5 百度

3.5.1 实现一个带并发限制的异步调度器 Scheduler，保证同时运行的任务最多有N个。完善下面代码中的 Scheduler 类，使得以下程序能正确输出：

实现一个带并发限制的异步调度器 Scheduler，保证同时运行的任务最多有N个。完善下面代码中的 Scheduler 类，使得以下程序能正确输出：

Code block

```
1  class Scheduler {
2    add(promiseCreator) { ... }
3    // ...
4  }
5
6  const timeout = (time) => new Promise(resolve => {
7    setTimeout(resolve, time)
8  })
9
10 const scheduler = new Scheduler(n)
11 const addTask = (time, order) => {
12   scheduler.add(() => timeout(time)).then(() => console.log(order))
13 }
```

```
14
15   addTask(1000, '1')
16   addTask(500, '2')
17   addTask(300, '3')
18   addTask(400, '4')
19
20   // 打印顺序是: 2 3 1 4
```

流程分析:

整个的完整执行流程:

1. 起始1、2两个任务开始执行;
2. 500ms时, 2任务执行完毕, 输出2, 任务3开始执行;
3. 800ms时, 3任务执行完毕, 输出3, 任务4开始执行;
4. 1000ms时, 1任务执行完毕, 输出1, 此时只剩下4任务在执行;
5. 1200ms时, 4任务执行完毕, 输出4;

当资源不足时将任务加入等待队列, 当资源足够时, 将等待队列中的任务取出执行

在调度器中一般会有一个等待队列queue, 存放当资源不够时等待执行的任务

具有并发数据限制, 假设通过max设置允许同时运行的任务, 还需要count表示当前正在执行的任务数量

当需要执行一个任务A时, 先判断count==max 如果相等说明任务A不能执行, 应该被阻塞, 阻塞的任务放进queue中, 等待任务调度器管理

如果count<max说明正在执行的任务数没有达到最大容量, 那么count++执行任务A, 执行完毕后count--

此时如果queue中有值, 说明之前有任务因为并发数量限制而被阻塞, 现在count<max, 任务调度器会将队头的任务弹出执行

Code block

```
1   class Scheduler {
2     constructor(max) {
3       this.max = max;
4       this.count = 0; // 用来记录当前正在执行的异步函数
5       this.queue = new Array(); // 表示等待队列
6     }
7     async add(promiseCreator) {
8       /*
9        * 此时count已经满了, 不能执行本次add需要阻塞在这里, 将resolve放入队列中等待唤醒,
10        * 等到count<max时, 从队列中取出执行resolve, 执行, await执行完毕, 本次add继续
11        */
12       if (this.count >= this.max) {
13         await new Promise((resolve, reject) => this.queue.push(resolve));
14       }
15
16       this.count++;
17       let res = await promiseCreator();
18       this.count--;
19       if (this.queue.length) {
20         // 依次唤醒add
21         // 若队列中有值, 将其resolve弹出, 并执行
22         // 以便阻塞的任务, 可以正常执行
23         this.queue.shift();
24       }
25       return res;
26     }
27   }
28
29   const timeout = time =>
30     new Promise(resolve => {
31       setTimeout(resolve, time);
```

```
32     });
33
34     const scheduler = new Scheduler(2);
35
36     const addTask = (time, order) => {
37         //add返回一个promise, 参数也是一个promise
38         scheduler.add(() => timeout(time)).then(() => console.log(order));
39     };
40
41     addTask(1000, '1');
42     addTask(500, '2');
43     addTask(300, '3');
44     addTask(400, '4');
45
46     // output: 2 3 1 4
```

4. 具体技术栈应该掌握到什么程度？



EncodeStudio
印客学院

印客学院2025匠心之作
内容&服务全面升级

WEB

Web前端

大厂工程师训练营

进阶前端架构
直击阿里P7



5. 前端算法面试题核心考查重点

5.1 双指针

5.1.1 接近的三数之和

给定一个包括 n 个整数的数组 `nums` 和一个目标值 `target`。找出 `nums` 中的三个整数，使得它们的和与 `target` 最接近。返回这三个数的和。假定每组输入只存在唯一答案。

Code block

```
1  示例：
2
3  输入：nums = [-1,2,1,-4]，target = 1
4  输出：2
5  解释：与 target 最接近的和是 2 (-1 + 2 + 1 = 2) 。
```

提示：

`3 <= nums.length <= 10^3`

`-10^3 <= nums[i] <= 10^3`

`-10^4 <= target <= 10^4`

链接：<https://leetcode-cn.com/problems/3sum-closest>

先按照升序排序，然后分别从左往右依次选择一个基础点 `i` (`0 <= i <= nums.length - 3`)，在基础点的右侧用双指针去不断的找最小的差值。

假设基础点是 `i`，初始化的时候，双指针分别是：

- `left`：`i + 1`，基础点右边一位。
- `right`：`nums.length - 1` 数组最后一位。

然后求此时的和，如果和大于 `target`，那么可以把右指针左移一位，去试试更小一点的值，反之则把左指针右移。

在这个过程中，不断更新全局的最小差值 `min`，和此时记录下来的和 `res`。

最后返回 `res` 即可。

Code block

```
1  /**
2   * @param {number[]} nums
3   * @param {number} target
4   * @return {number}
5   */
6  let threeSumClosest = function (nums, target) {
7      let n = nums.length
8      if (n === 3) {
9          return getSum(nums)
10     }
11     // 先升序排序 此为解题的前置条件
12     nums.sort((a, b) => a - b)
13
14     let min = Infinity // 和 target 的最小差
15     let res
16
17     // 从左往右依次尝试定一个基础指针 右边至少再保留两位 否则无法凑成3个
18     for (let i = 0; i <= nums.length - 3; i++) {
19         let basic = nums[i]
20         let left = i + 1; // 左指针先从 i 右侧的第一位开始尝试
21         let right = n - 1 // 右指针先从数组最后一项开始尝试
```

```

22
23     while (left < right) {
24         let sum = basic + nums[left] + nums[right] // 三数求和
25         // 更新最小差
26         let diff = Math.abs(sum - target)
27         if (diff < min) {
28             min = diff
29             res = sum
30         }
31         if (sum < target) {
32             // 求出的和如果小于目标值的话 可以尝试把左指针右移 扩大值
33             left++
34         } else if (sum > target) {
35             // 反之则右指针左移
36             right--
37         } else {
38             // 相等的话 差就为0 一定是答案
39             return sum
40         }
41     }
42 }
43
44 return res
45 };
46
47 function getSum(nums) {
48     return nums.reduce((total, cur) => total + cur, 0)
49 }

```

5.1.2 通过删除字母匹配到字典里最长单词

给定一个字符串和一个字符串字典，找到字典里面最长的字符串，该字符串可以通过删除给定字符串的某些字符来得到。如果答案不止一个，返回长度最长且字典顺序最小的字符串。如果答案不存在，则返回空字符串。

Code block

```

1  示例 1:
2
3  输入:
4  s = "abpcplea", d = ["ale","apple","monkey","plea"]
5
6  输出:
7  "apple"
8  示例 2:
9
10 输入:
11 s = "abpcplea", d = ["a","b","c"]
12
13 输出:
14 "a"
15 说明:
16
17 所有输入的字符串只包含小写字母。
18 字典的大小不会超过 1000。
19 所有输入的字符串长度不会超过 1000。

```

链接: <https://leetcode-cn.com/problems/longest-word-in-dictionary-through-deleting>

为数组 `d` 中的**每个**单词分别生成**指针**，指向那个单词目前**已经匹配到**的字符下标。然后对 `s` 进行一次遍历，每轮循环中都对 `d` 中所有指针的**下一位**进行匹配，如果匹配到了，则那个单词的指针后移。

一旦某个单词指针移动到那个字母的最后一位了，就和全局的 `find` 变量比较，先比较长度，后比较字典序，记录下来，最后返回即可。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string[]} d
4   * @return {string}
5   */
6  let findLongestWord = function (s, d) {
7      let n = d.length
8      let points = Array(n).fill(-1)
9
10     let find = ""
11     for (let i = 0; i < s.length; i++) {
12         let char = s[i]
13         for (let j = 0; j < n; j++) {
14             let targetChar = d[j][points[j] + 1]
15             if (char === targetChar) {
16                 points[j]++
17                 let word = d[j]
18                 let wl = d[j].length
19                 if (points[j] === wl - 1) {
20                     let fl = find.length
21                     if (wl > fl || (wl === fl && word < find)) {
22                         find = word
23                     }
24                 }
25             }
26         }
27     }
28
29     return find
30 }
```

5.1.3 判断子序列

给定字符串 `s` 和 `t`，判断 `s` 是否为 `t` 的子序列。

你可以认为 `s` 和 `t` 中仅包含英文小写字母。字符串 `t` 可能会很长（长度 $\sim 500,000$ ），而 `s` 是个短字符串（长度 ≤ 100 ）。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，"ace"是"abcde"的一个子序列，而"aec"不是）。

Code block

```
1  示例 1:
2  s = "abc", t = "ahbgdc"
3
4  返回 true.
5
6  示例 2:
7  s = "axc", t = "ahbgdc"
8
9  返回 false.
```

后续挑战：

如果有大量输入的 S ，称作 $S_1, S_2, \dots, S_k$ 其中 $k \geq 10$ 亿，你需要依次检查它们是否为 T 的子序列。在这种情况下，你会怎样改变代码？

链接：<https://leetcode-cn.com/problems/is-subsequence>

判断字符串 s 是否是字符串 t 的子序列，我们可以建立一个 i 指针指向「当前 s 已经在 t 中成功匹配的字符下标后一位」。之后开始遍历 t 字符串，每当在 t 中发现 i 指针指向的目标字符时，就可以把 i 往后前进一位。

- 一旦 `i === t.length`，就代表 t 中的字符串全部按顺序在 s 中找到了，返回 `true`。
- 当遍历 s 结束后，就返回 `false`，因为 i 此时并没有成功走 t 的最后一位。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string} t
4   * @return {boolean}
5   */
6  let isSubsequence = function (s, t) {
7      let sl = s.length
8      if (!sl) {
9          return true
10     }
11
12     let i = 0
13     for (let j = 0; j < t.length; j++) {
14         let target = s[i]
15         let cur = t[j]
16         if (cur === target) {
17             i++
18             if (i === sl) {
19                 return true
20             }
21         }
22     }
23
24     return false
25 };
```

5.1.4 分发饼干

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。对每个孩子 i ，都有一个胃口值 g_i ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干 j ，都有一个尺寸 s_j 。如果 $s_j \geq g_i$ ，我们可以将这个饼干 j 分配给孩子 i ，这个孩子会得到满足。你的目标是尽可能满足越多数量的孩子，并输出这个最大数值。

注意：

你可以假设胃口值为正。

一个小朋友最多只能拥有一块饼干。

Code block

```
1  示例 1:
2
3  输入: [1,2,3], [1,1]
4
5  输出: 1
6
7  解释:
8  你有三个孩子和两块小饼干，3个孩子的胃口值分别是：1,2,3。
9  虽然你有两块小饼干，由于他们的尺寸都是1，你只能让胃口值是1的孩子满足。
```

```
10  所以你应该输出1。
11  示例 2:
12
13  输入: [1,2], [1,2,3]
14
15  输出: 2
16
17  解释:
18  你有两个孩子和三块小饼干, 2个孩子的胃口值分别是1,2。
19  你拥有的饼干数量和尺寸都足以让所有孩子满足。
20  所以你应该输出2。
```

链接: <https://leetcode-cn.com/problems/assign-cookies>

把饼干和孩子的需求都排序好, 然后从最小的饼干分配给需求最小的孩子开始, 不断的尝试新的饼干和新的孩子, 这样能保证每个分给孩子的饼干都恰到好处的不浪费, 又满足需求。

利用双指针不断的更新 `i` 孩子的需求下标和 `j` 饼干的值, 直到两者有其一达到了终点位置:

1. 如果当前的饼干不满足孩子的胃口, 那么把 `j++` 寻找下一个饼干, 不用担心这个饼干被浪费, 因为这个饼干更不可能满足下一个孩子 (胃口更大)。
2. 如果满足, 那么 `i++; j++; count++` 记录当前的成功数量, 继续寻找下一个孩子和下一个饼干。

Code block

```
1  /**
2   * @param {number[]} g
3   * @param {number[]} s
4   * @return {number}
5   */
6  let findContentChildren = function (g, s) {
7      g.sort((a, b) => a - b)
8      s.sort((a, b) => a - b)
9
10     let i = 0
11     let j = 0
12
13     let count = 0
14     while (j < s.length && i < g.length) {
15         let need = g[i]
16         let cookie = s[j]
17
18         if (cookie >= need) {
19             count++
20             i++
21             j++
22         } else {
23             j++
24         }
25     }
26
27     return count
28 }
```

5.1.5 验证回文串

给定一个字符串, 验证它是否是回文串, 只考虑字母和数字字符, 可以忽略字母的大小写。

说明: 本题中, 我们将空字符串定义为有效的回文串。

示例 1:

Code block

```
1  输入: "A man, a plan, a canal: Panama"
2  输出: true
```

示例 2:

Code block

```
1  输入: "race a car"
2  输出: false
```

<https://leetcode-cn.com/problems/valid-palindrome>

先根据题目给出的条件，通过正则把不匹配字符去掉，然后转小写。

建立双指针 `i`，`j` 分别指向头和尾，然后两个指针不断的向中间靠近，每前进一步就对比两端的字符串是否相等，如果不相等则直接返回 `false`。

如果直到 `i >= j` 也就是指针对撞了，都没有返回 `false`，那就说明符合「回文」的定义，返回 `true`。

Code block

```
1  /**
2   * @param {string} s
3   * @return {boolean}
4   */
5  let isPalindrome = function(s) {
6      s = s.replace(/[^0-9a-zA-Z]/g, '').toLowerCase()
7      let i = 0
8      let j = s.length - 1
9
10     while(i < j) {
11         let head = s[i]
12         let tail = s[j]
13
14         if (head !== tail) {
15             return false
16         } else {
17             i++
18             j--
19         }
20     }
21     return true
22 };
```

5.2 滑动窗口

5.2.1 滑动窗口的最大值

给定一个数组 `nums` 和滑动窗口的大小 `k`，请找出所有滑动窗口里的最大值。

示例:

Code block

```
1  输入: nums = [1,3,-1,-3,5,3,6,7], 和 k = 3
2  输出: [3,3,5,5,6,7]
```

3	解释：									
4										
5	滑动窗口的位置					最大值				
6	-----					-----				
7	[1	3	-1]	-3	5	3	6	7		3
8	1	[3	-1	-3]	5	3	6	7		3
9	1	3	[-1	-3	5]	3	6	7		5
10	1	3	-1	[-3	5	3]	6	7		5
11	1	3	-1	-3	[5	3	6]	7		6
12	1	3	-1	-3	5	[3	6	7]		7

提示：

你可以假设 k 总是有效的，在输入数组不为空的情况下， $1 \leq k \leq$ 输入数组的大小。

链接：<https://leetcode-cn.com/problems/hua-dong-chuang-kou-de-zui-da-zhi-lcof>

滑动窗口，每次左右各滑动一位，并且求窗口中的最大值记录即可，这题坑的地方在于边界情况比较多。

Code block

```
1  let maxSlidingWindow = function (nums, k) {
2    if (k === 0 || !nums.length) {
3      return []
4    }
5    let left = 0
6    let right = k - 1
7    let res = [findMax(nums, left, right)]
8
9    while (right < nums.length - 1) {
10     right++
11     left++
12     res.push(findMax(nums, left, right))
13   }
14
15   return res
16 }
17
18 function findMax(nums, left, right) {
19   let max = -Infinity
20   for (let i = left; i <= right; i++) {
21     max = Math.max(max, nums[i])
22   }
23   return max
24 }
```

5.2.2 找到字符串中所有字母异位词

给定一个字符串 s 和一个非空字符串 p，找到 s 中所有是 p 的字母异位词的子串，返回这些子串的起始索引。

字符串只包含小写英文字母，并且字符串 s 和 p 的长度都不超过 20100。

说明：

字母异位词指字母相同，但排列不同的字符串。

不考虑答案输出的顺序。

示例 1:

Code block

```
1  输入：
```



```
2  s: "cbaebabacd" p: "abc"
3
4  输出:
5  [0, 6]
6
7  解释:
8  起始索引等于 0 的子串是 "cba", 它是 "abc" 的字母异位词。
9  起始索引等于 6 的子串是 "bac", 它是 "abc" 的字母异位词。
```

示例 2:

Code block

```
1  输入:
2  s: "abab" p: "ab"
3
4  输出:
5  [0, 1, 2]
6
7  解释:
8  起始索引等于 0 的子串是 "ab", 它是 "ab" 的字母异位词。
9  起始索引等于 1 的子串是 "ba", 它是 "ab" 的字母异位词。
10 起始索引等于 2 的子串是 "ab", 它是 "ab" 的字母异位词。
```

链接: <https://leetcode-cn.com/problems/find-all-anagrams-in-a-string>

还是典型的滑动窗口去解决的问题，由于字母异位词一定是长度相等的，所以我们需要把窗口的长度始终维持在目标字符的长度，也就是说，每次循环结束后 `left` 和 `right` 是同步前进的。

由于字母异位词不需要考虑顺序，所以只需要运用一个辅助函数 `isAnagrams` 去判断两个 **map** 中记录的字母次数，即可判断出当前位置开始的子串是否和目标字符串形成字母异位词。

Code block

```
1  /**
2   * @param {string} s
3   * @param {string} p
4   * @return {number[]}
5   */
6  let findAnagrams = function (s, p) {
7    let targetMap = makeCountMap(p)
8    let sl = s.length
9    let pl = p.length
10   // [left,...right] 滑动窗口
11   let left = 0
12   let right = pl - 1
13   let windowMap = makeCountMap(s.substring(left, right + 1))
14   let res = []
15
16   while (left <= sl - pl && right < sl) {
17     if (isAnagrams(windowMap, targetMap)) {
18       res.push(left)
19     }
20     windowMap[s[left]]--
21     right++
22     left++
23     addCountToMap(windowMap, s[right])
24   }
25 }
```

```

26     return res
27 }
28
29 let isAnagrams = function (windowMap, targetMap) {
30     let targetKeys = Object.keys(targetMap)
31     for (let targetKey of targetKeys) {
32         if (
33             !windowMap[targetKey] ||
34             windowMap[targetKey] !== targetMap[targetKey]
35         ) {
36             return false
37         }
38     }
39     return true
40 }
41
42 function addCountToMap(map, str) {
43     if (!map[str]) {
44         map[str] = 1
45     } else {
46         map[str]++
47     }
48 }
49
50 function makeCountMap(strs) {
51     let map = {}
52     for (let i = 0; i < strs.length; i++) {
53         let letter = strs[i]
54         addCountToMap(map, letter)
55     }
56     return map
57 }

```

5.2.3 最小覆盖子串

给你一个字符串 S、一个字符串 T，请在字符串 S 里面找出：包含 T 所有字符的最小子串。

示例：

Code block

```

1  输入：S = "ADOBECODEBANC", T = "ABC"
2  输出："BANC"

```

说明：

如果 S 中不存这样的子串，则返回空字符串 ""。

如果 S 中存在这样的子串，我们保证它是唯一的答案。

<https://leetcode-cn.com/problems/minimum-window-substring>

根据目标字符串 `t` 生成一个目标 map，记录每个字符的目标值出现的次数。

然后就是维护一个滑动窗口，并且针对这个滑动窗口中的字符也生成一个 map 去记录字符出现次数。

每次循环都去对比窗口的 map 里的字符是否能覆盖目标 map 里的字符。

覆盖的意思就是，目标 map 里的每个字符在窗口 map 中出现，并且出现的次数要 $\geq$ 目标 map 中此字符出现的次数。

窗口滑动逻辑：

1. 如果当前还没有能覆盖，那么就右滑右边界。

2. 如果当前已经覆盖了，记录下当前的子串，并且右滑左边界看看能否进一步缩小子串的长度。

两种情况下停止循环，返回结果：

1. 左边界达到 给定字符长度 - 目标字符的长度，此时不管匹配与否，都是最短能满足的了。
2. 右边界超出给定字符的长度，这种情况会出现在右边界已经到头了，但是还没有能覆盖目标字符串，此时就算继续滑动也不可能得到结果了。

Code block

```
1  var findAnagrams = function(s, p) {
2    let sL = s.length, pL = p.length
3    if (sL < pL) return [];
4
5    let left = 0, right = 0, // 滑动窗口两侧索引
6        windowMap = new Map(), // 统计滑动窗口内每个 char 的数量
7        pMap = new Map(), // 统计字符串 p 内每个 char 的数量
8        valid = 0, // 滑动窗口 和 字符串 p 中数量相等的 char 的个数
9        ret = [] // 结果集
10
11    for (const char of p) {
12        pMap.set(char, (pMap.get(char) || 0) + 1)
13    }
14
15    while (right < sL) {
16        let char = s[right++]
17        if (pMap.has(char)) {
18            windowMap.set(char, (windowMap.get(char) || 0) + 1);
19            (windowMap.get(char) === pMap.get(char)) && valid++
20        }
21
22        if (right - left === pL) {
23            // 滑动窗口内的字符数和 pL 相等时，并且 2 个 map 中每个 char 的数量都相等，说明找到异位词了。
24            valid === pMap.size && ret.push(left)
25
26            // 无论有没有找到异位词，left 都要右移了。
27            let char = s[left++]
28            if (pMap.has(char)) {
29                // 如果 pMap 中有 char，并且 pMap 中 char 的数量和 windowMap 中的相等，（因为 left 要右移了，所以）
                valid--,
30                (windowMap.get(char) === pMap.get(char)) && valid--;
31                // 因为 left 要右移了，所以 windowMap 中的 char 数量也 - 1，
32                windowMap.set(char, windowMap.get(char) - 1);
33            }
34        }
35    }
36    return ret
37  };
```

6. 师资&课程内容

1. 师资：一线大厂/外企现役P8级高级专家亲自带；
2. 内容：深度对标阿里P6-P7，涵盖大厂所有内训标准；

6.1 项目实战内容

github: <https://github.com/encode-studio-fe>

[前端实战课分享汇总](#)（可咨询班主任要密码权限）

6.2 服务内容

系统&突击两手抓

全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



6.3 专业一对一

近期一对一

- 📅 (2025-02-06) xxx-2.5年-React-24K
- 📅 (2025-02-07) xxx-1年-React-15k
- 📅 (2025-01-05) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历（For 付费学员）

6.4 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

6.5 近期报名学员案例

